



**Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro**



Lab Session 4

Task 1: The Identity Setup

Objective: Configure your global Git username and email address. Before saving any work, Git needs to know who you are so it can label your code changes accurately.

Instructions: Open your terminal or command prompt and run the following two commands (replace the placeholder text with your actual name and email):

```
git config --global user.name "Your Full Name"  
git config --global user.email "your.email@example.com"
```

To verify that Git has successfully saved your identity details, run:

```
git config --list
```

Task 2: Building Your First Code Vault

Objective: Initialize a brand-new, empty Git repository inside a project folder to start tracking files.

Instructions:

1. Create a new folder on your computer named `python_lab_four`.
2. Open your terminal, navigate into that folder, and initialize it as a Git repository using the following command:

```
git init
```

Hint: Look closely at your terminal after running this command. It will print a hidden message confirming that it has initialized an empty Git repository in your directory.



Task 3: The Project Log Checklist

Objective: Check the current state of your folder using the status tool to see which files Git is watching.

Instructions:

Run the status command in your terminal:

```
git status
```

Expected Output Notice: Because this folder is completely brand new and empty, Git will print a message stating nothing to commit, working tree clean or indicate that there are no files tracking yet.

Task 4: Staging Your Python Script

Objective: Create a Python file, write a simple program, and explicitly add it to Git's "staging area" (the preparation zone before saving).

Instructions:

1. Inside your `python_lab_four` folder, create a file named `app.py`.
2. Open `app.py` in your text editor and write a quick one-line script:

```
print("My first Git tracked application!")
```

3. Go back to your terminal and run `git status`. You will notice `app.py` is listed in red text under Untracked files.
4. Move the file into the staging area by running:

```
git add app.py
```

Hint: Run `git status` once more after adding it. The file name will change from red to green, showing it is now safely sitting in the staging area.



Task 5: Making the First Permanent Save

Objective: Permanently save your staged changes into your Git history by creating a "commit" with a descriptive message.

Instructions:

Commit your staged file to the repository's permanent timeline by running the following command:

```
git commit -m "Initial commit: Add app.py script"
```

Task 6: Reviewing the History Timeline

Objective: View the historical log of all saved commits made inside your repository.

Instructions:

To see your newly created commit, along with its unique tracking ID hash, author identity, timestamp, and message, run:

```
git log
```

Hint: If your log history becomes long and fills the screen, you can press the q key on your keyboard to exit back to your regular terminal prompt safely.

Task 7: Tracking Code Evolutions

Objective: Modify an existing file, observe how Git tracks changes to existing code, and commit a second version.

Instructions:

1. Re-open your app.py file and add a second line of code to it:

```
print("My first Git tracked application!")  
print("This line represents version 2.0 of my script.")
```

2. Run git status. Git will intelligently flag that app.py has been Modified since your last save.
3. Stage and commit this change sequentially using these commands:



**Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro**



```
git add app.py  
git commit -m "Update app.py with version 2 print statement"
```

Task 8: The Compact Log Layout

Objective: Display a clean, multi-line condensed list layout of your project timeline using advanced log formatting flags.

Instructions:

Print out your entire project history neatly on single lines to view both of your milestones clearly by running:

```
git log --oneline
```

Task 9: Tracking Document Mistakes

Objective: Intentionally cause a tracking discrepancy to learn how Git highlights exact line edits.

Instructions:

1. Modify your app.py file one last time by adding a third line: `print("Debugging git operations")`.
2. Before staging it, run the difference tracking command:

```
git diff
```

Hint: Look at the terminal output lines marked with a + sign. Git uses these signs and green text to highlight exactly what you added to the file since your last commit.

Task 10: Mini Git Cheat-Sheet Generator

Objective: Create a permanent multi-line text file summary inside your vault documenting the key commands you learned today.

Instructions:

1. Create a file named cheatsheet.txt.



**Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro**



2. Add the following text exactly as it looks inside it, preserving the line breaks:

```
=====
GIT BASE CHEATSHEET
=====
1. git init          -> Start repo
2. git status        -> Check state
3. git add <file>    -> Stage files
4. git commit -m ""  -> Save snapshot
5. git log --oneline -> View history
=====
```

3. Run the final sequence to stage and save your cheat sheet permanently into your local history log:

```
git add cheatsheet.txt
git commit -m "Add a local git cheat sheet text file"
git log --oneline
```